

Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify

## { T }
```

Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, imitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y is between 0 and 1, which represents 16,000 years ago to 2,400 years ago.
myFunc <- function(x) {
  case_when(
    0 <= x & x <= (-9+ 16)/(13.6) ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x & x <= (-7.500 + 16)/(13.6) ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x & x <= (-4.3+ 16)/(13.6) ~ 3,
    (-4.3 + 16)/(13.6) < x & x <= (-3.000+ 16)/(13.6) ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x & x <= (-2.400+ 16)/(13.6) ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE ~ 1
  )
}
curve(myFunc(x), col = 2, main = "Relative Sea Level and Year")

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
  T[i]<-rpois(1, Y[i]* 25)
  if(T[i] < 1) {T[i]<- 1}
  else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
  SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL, main = "Synthetic Data Plot(T and SL)")
```

```

plot(Y, T, main = "Synthetic Data Plot(Y and T)")
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5

## [1] 0.1468741 0.4928763

plot(density(S), main = "Density of Synthetic Data of S")

plot(dS, S, main = "Synthetic Data Plot(dS and S) \n with different groups")
legend(x = 0, y = 0.18, legend="red: T = 2 blue: T = 7 green: T = 13")
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S, main = "Synthetic Data Plot(T and S)")
plot(Y, S, main = "Synthetic Data Plot(Y and S)")
plot(T, dS, main = "Synthetic Data Plot(T and dS)\n No correlation expected.") # no correlation
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i] - dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K) # try to make less than 0.5

## [1] 0.08423709 0.43875485

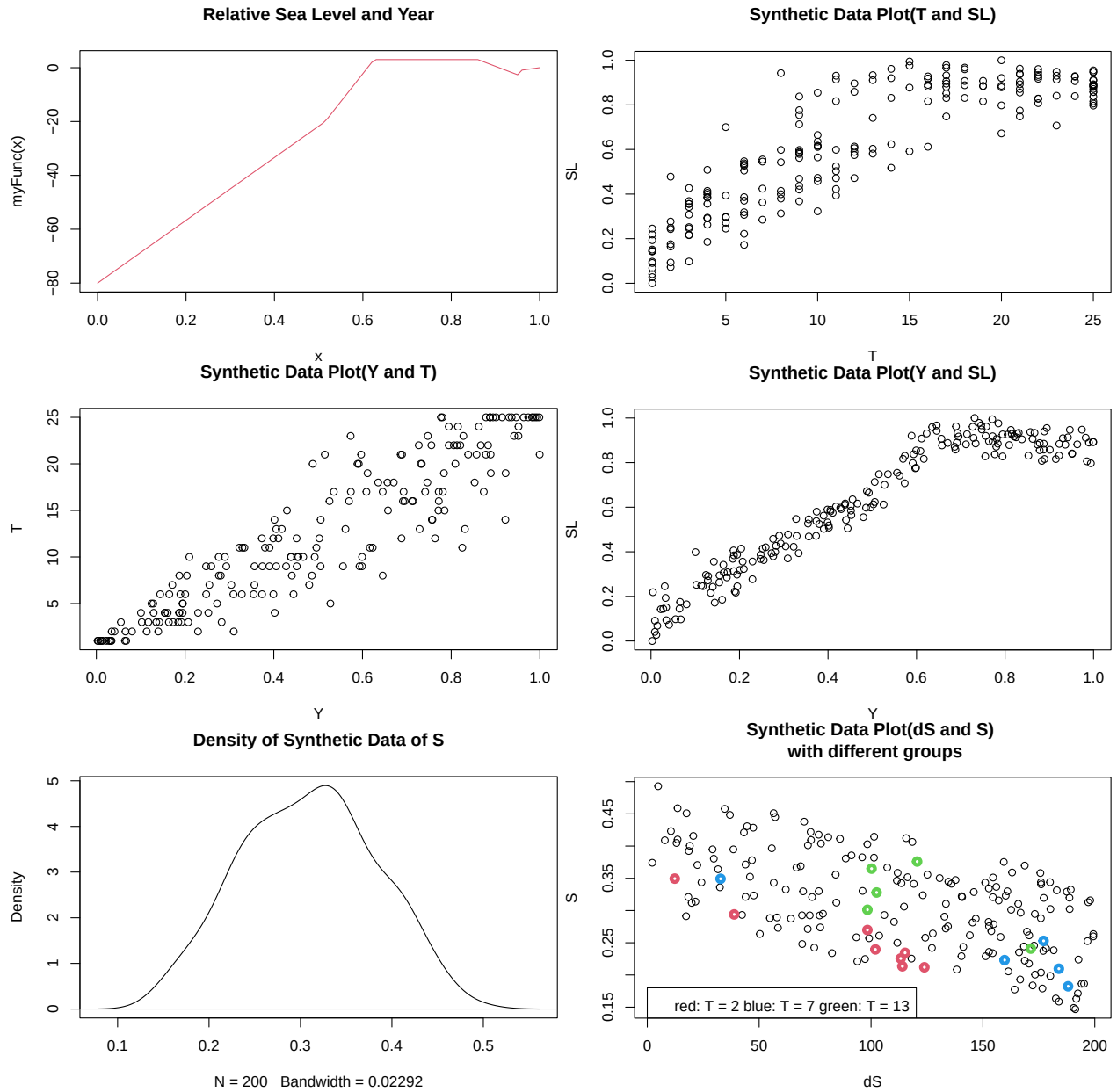
plot(density(K), main = "Density of Synthetic Data of K")
plot(dK, K, main = "Synthetic Data Plot(dK and K)")
plot(SL, K, main = "Synthetic Data Plot(SL and K)")
plot(Y, K, main = "Synthetic Data Plot(Y and K)")
plot(S, K, main = "Synthetic Data Plot(S and K)")
plot(dS, K, main = "Synthetic Data Plot(dS and K)\n No correlation expected.") # no correlation

# Standardize data without SL, S, K (those are ratio)
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))

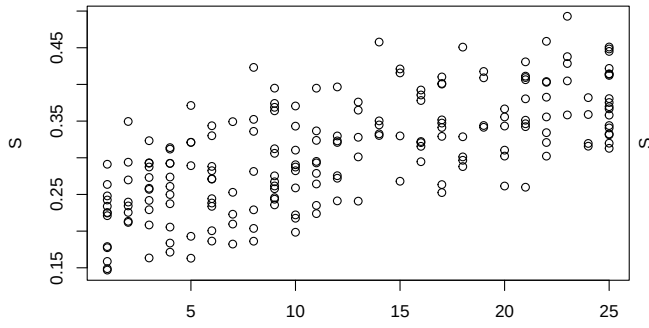
# check the data
plot(density(d$K), main = "Density of Synthetic Data of d$K")
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
plot(Y, d$K, main = "Synthetic Data Plot(Y and d$K)")
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")

```

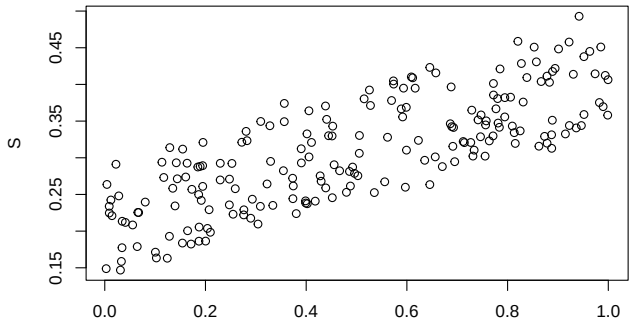
```
plot(Y, d$S, main = "Synthetic Data Plot(Y and d$S)")
plot(d$dS, d$K, main = "Synthetic Data Plot(d$dS and d$K)\n No correlation expected.") # no correlation
```



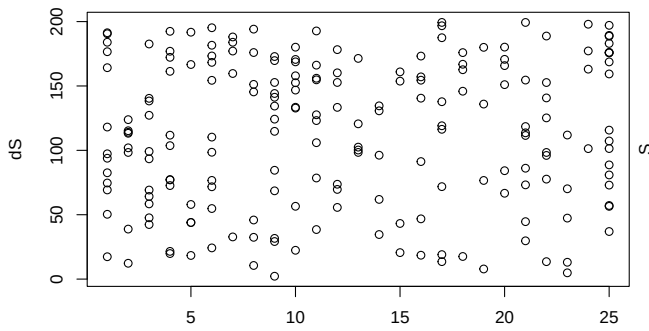
Synthetic Data Plot(T and S)



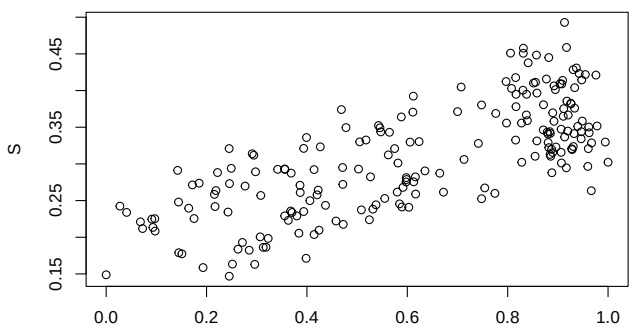
Synthetic Data Plot(Y and S)



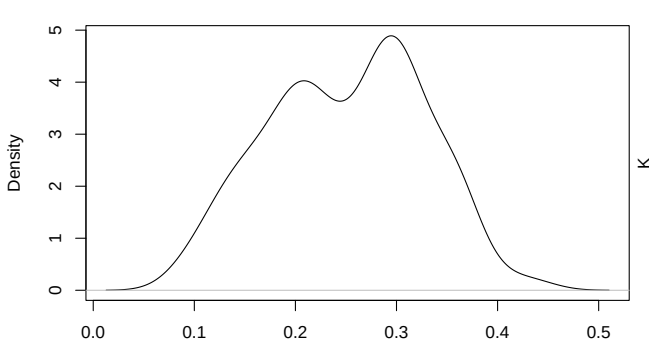
Synthetic Data Plot(T and dS)
No correlation expected.



Synthetic Data Plot(SL and S)
auto-correlation

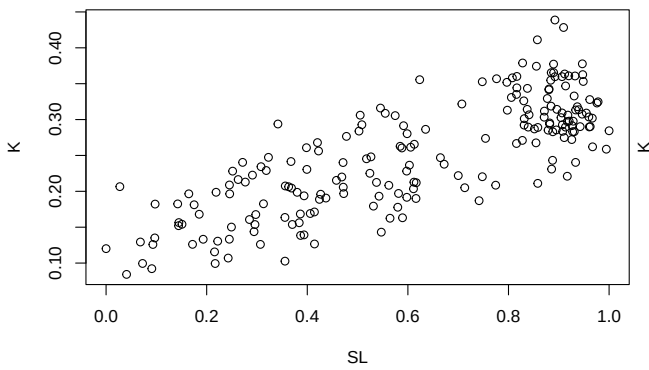


Density of Synthetic Data of K

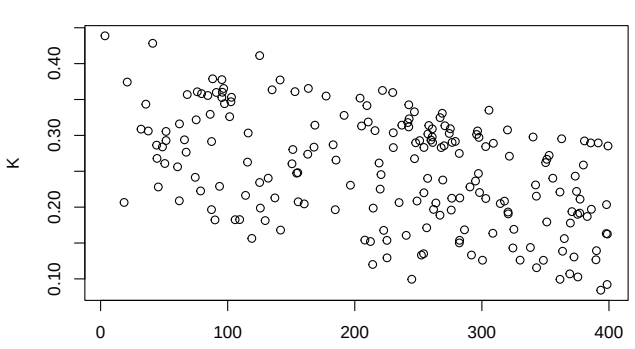


N = 200 Bandwidth = 0.02385

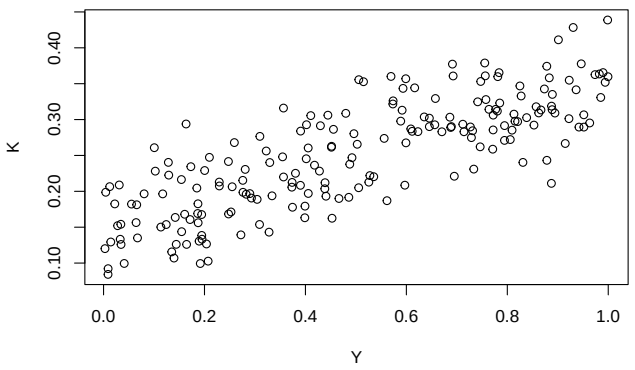
Synthetic Data Plot(SL and K)



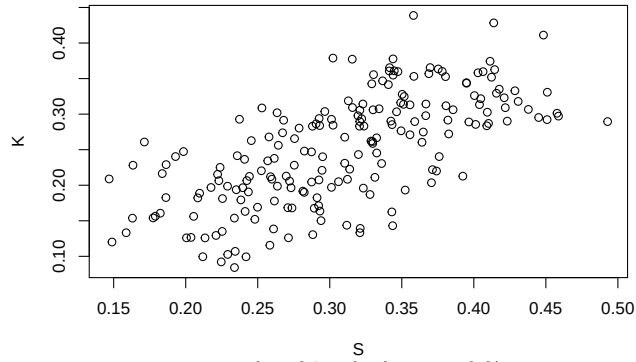
Synthetic Data Plot(dK and K)



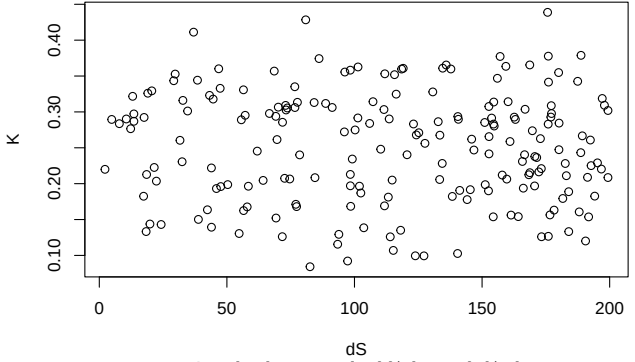
Synthetic Data Plot(Y and K)



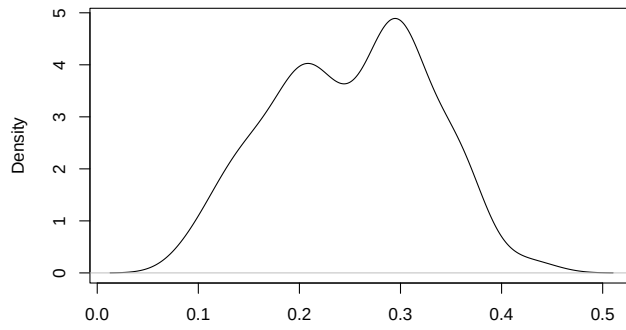
Synthetic Data Plot(S and K)



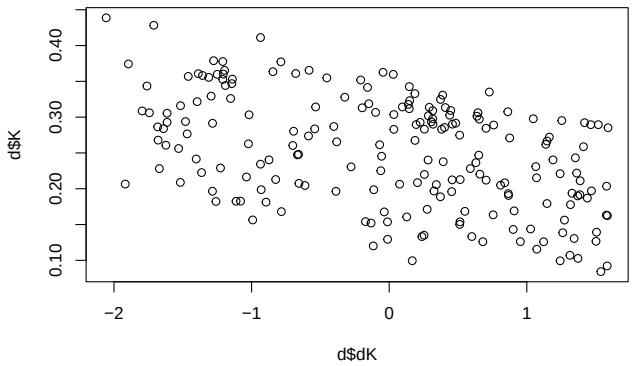
Synthetic Data Plot(dS and K)
No correlation expected.



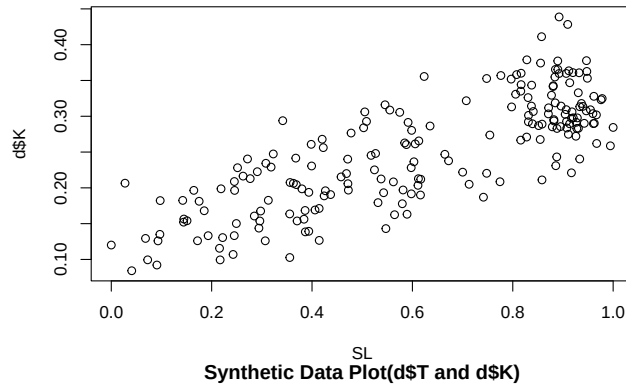
Density of Synthetic Data of d\$K



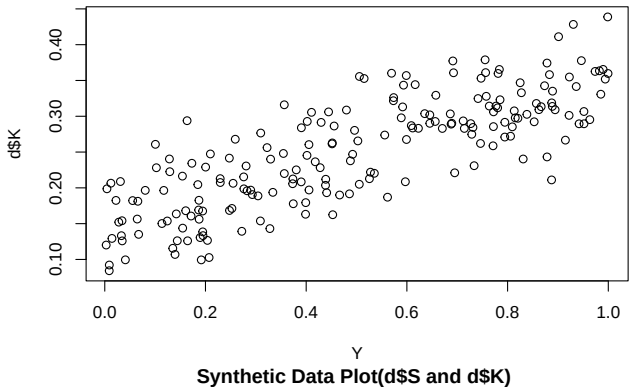
Synthetic Data Plot(d\$K and d\$K)



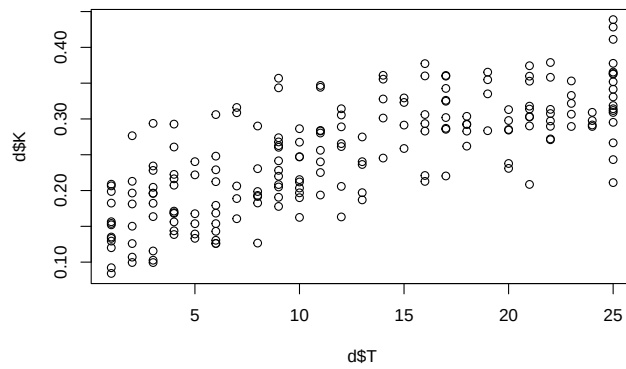
Synthetic Data Plot(SL and d\$K)



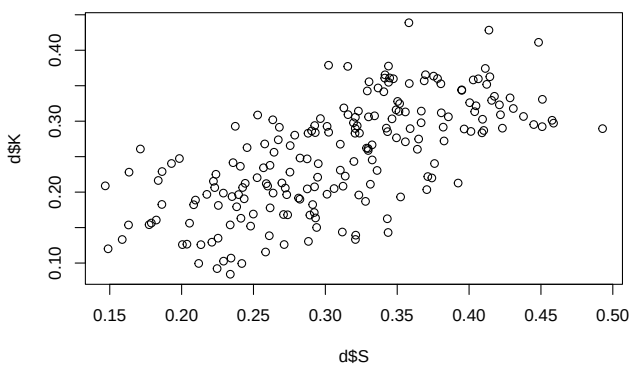
Synthetic Data Plot(Y and d\$K)

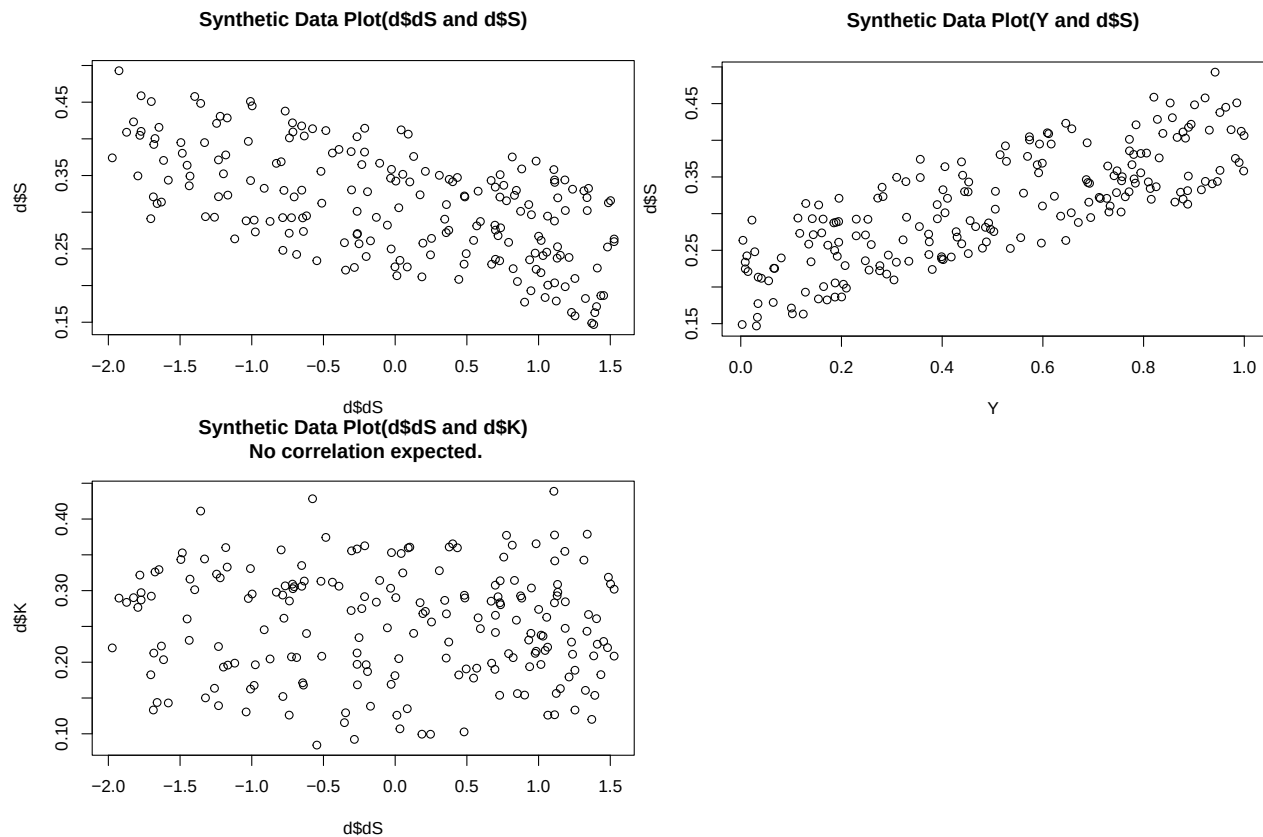


Synthetic Data Plot(d\$T and d\$K)



Synthetic Data Plot(d\$S and d\$K)

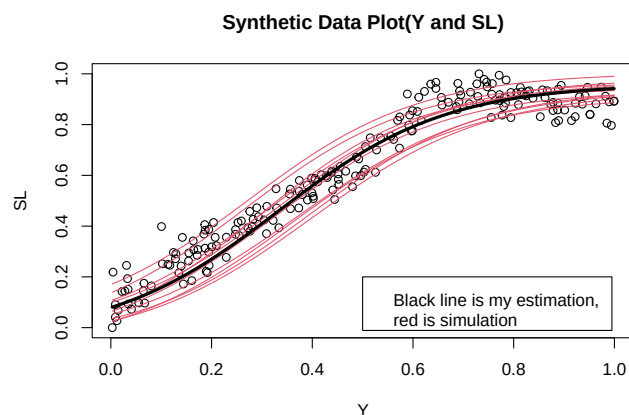




Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and SL are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 6, 0.2)*x + rnorm(1, -2, 0.2)) - rnorm(1, 0.04, 0.02), add =TRUE)
curve(inv_logit(6*x - 2) - 0.04, add =TRUE, lwd = 3)
legend(0.5, 0.2, legend="Black line is my estimation, \nred is simulation")
```



Model 1

- Also see overview.pdf to know statistical model and prior
- This model stratifies all parameters by T.
- There are a few messages to warning my priors. Since it only shows up during first iteration of warmup, we can safely ignore it.
- I just omit it for m1.2 and 1.3.

```
# Model 1
m1<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # We do not know anything about this but think similar for all group.
    # So use partial pooling to regularize the estimate.
    # mean should start at 0 and has small variation.

    # In this case (m1), we do not include correlation.
    # This is because multilevel models of S and SL includes the information of Y
    # But we have to regularize.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_S_bar, bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    # don't need regularization as dK is not affected by T
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS,

    # I am hoping/believing b_dS is negative and b_Y_S is positive
    # partial pooling for the same reason for a2 and b_Y_S
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    zbS_Y[T] ~ normal(0,0.1),

    # don't need regularization
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),
```

```

# Model SL by Y
# The log function and prior was referred to the previous research of Sea Level Change over time
# This function is abstract so I will have to actually stan code to provide accurate estimates
# ulam code in r does not allow us to hard code the conditional function.
# vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal(0.5, 0.3)

), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
  #, T = "lower=1,upper=25" #this is unnecessary since T is provided in the dat

), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1843:3: note: by 'void ulam_cm
## 1843 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1843:3: note: by 'void ulam_cm
## 1843 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note: by 'T_ ulam_cmds
## 1880 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note: by 'T_ ulam_cmds
## 1880 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note: by 'T_ ulam_cmds

```



```

## const'
## 1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_crtp<T_>::log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const' defined but not used [-Wunused-function]
## 159 |   inline math::var log_prob(std::vector<math::var>& theta,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note:   by 'T_ ulam_cmds'
## 1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note:   by 'T_ ulam_cmds')
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be ill-conditioned or the model may be misspecified.
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note:   by 'T_ ulam_cmds')
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be ill-conditioned or the model may be misspecified.
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 2
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504040636699.hpp:1880:3: note:   by 'T_ ulam_cmds')
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be ill-conditioned or the model may be misspecified.
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 4
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)

```

```

## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)

```

```

## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)

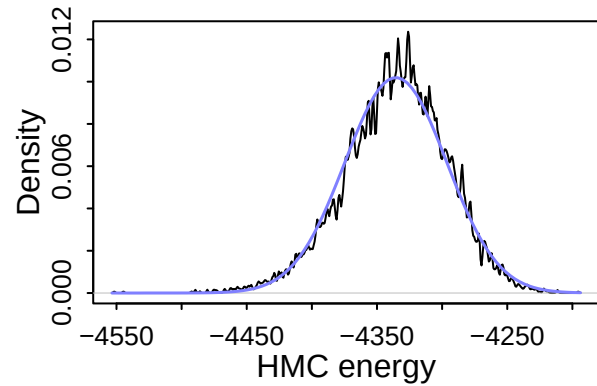
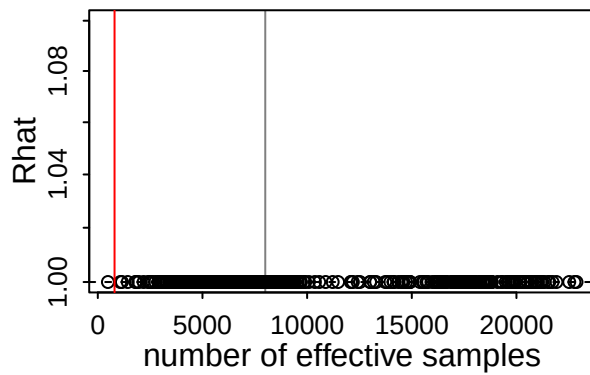
```

```

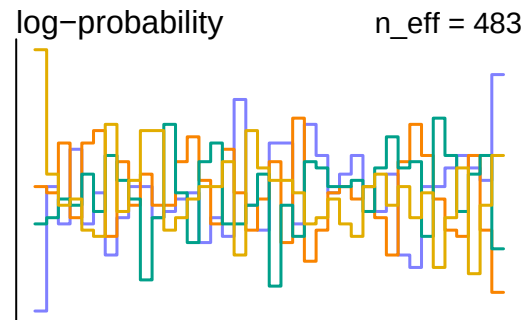
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 56.1 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 56.5 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 57.6 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 58.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 57.1 seconds.
## Total execution time: 58.6 seconds.

```

```
dashboard(m1)
```



0
Divergent transitions
Outlook good



- This has so much divergent and less effective sampling.
- This would be due to the large number of parameters, including hidden estimated ones (SL and Y). I will reduce parameters to reliably interpret the result.
 - First one would be Y as S and SL are more important for the model.
 - * by the way, SL and Y in model K change the efficiency in the highest rate.

```
# Model 1.2
m1.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
```

```

zbS_Y[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]: Y ~ normal( 0.5, 0.2)

), dat = d, constraints = list(
  SL = "lower=0,upper=1"
  , Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:  700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:  700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:  700 / 4000 [17%] (Warmup)

```

```

## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)

```

```

## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)

```



```

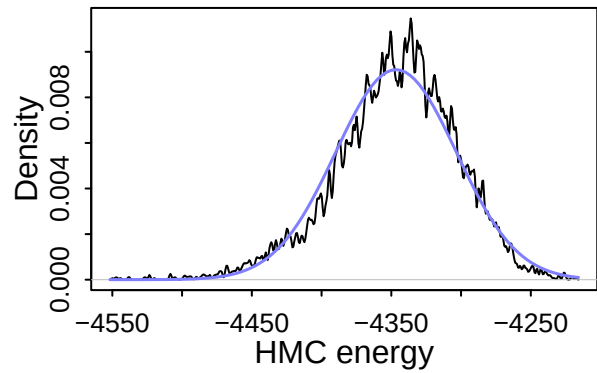
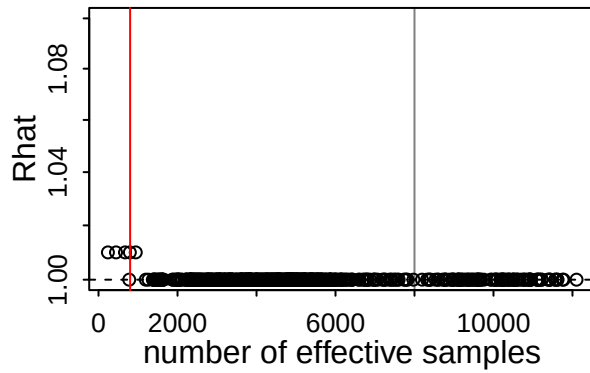
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 33.6 seconds.
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 33.9 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 35.3 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 36.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 34.7 seconds.
## Total execution time: 36.2 seconds.

```

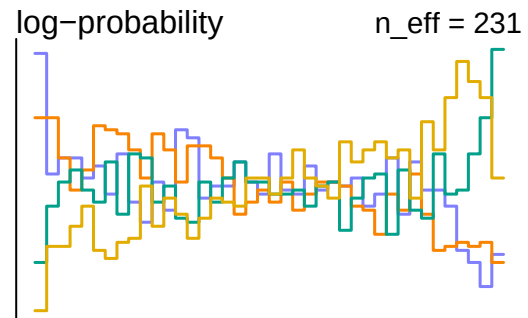
```

dashboard\(m1.2\)

```



0
Divergent transitions
Outlook good



This is still bad model as the eff is very small number

```
# Model 1.3
m1.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,

    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_dS * dS,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    aS_bar ~ normal(0,0.3),
    sigma_aS ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),

    # Model SL by Y
```

```

        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
        # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

        # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL      = "lower=0,upper=1",
        Y       = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)

```

```

## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)

```

```

## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)

```

```

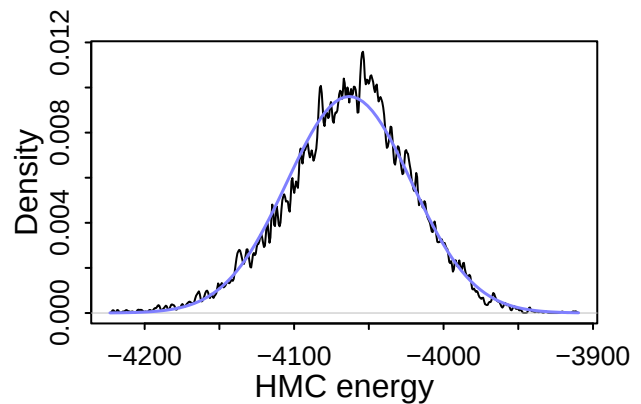
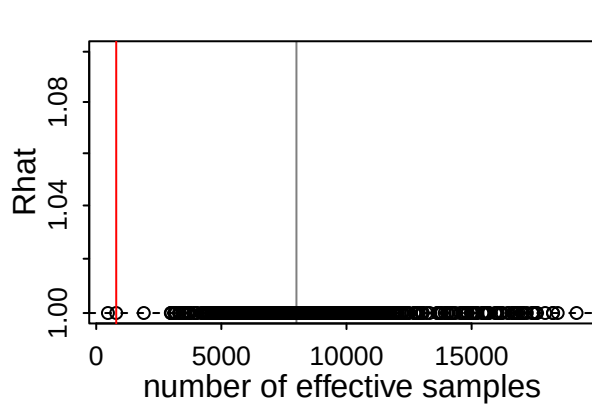
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 36.5 seconds.
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 37.5 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 39.3 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 39.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 38.2 seconds.
## Total execution time: 39.6 seconds.

```

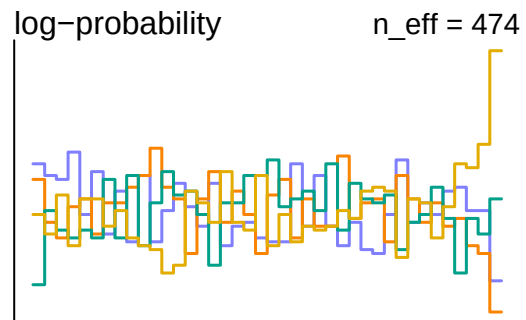
```

dashboard(m1.3)

```



0
Divergent transitions
Outlook good



```
precis(m1.3, depth=1)
```

	mean	sd	5.5%	94.5%	rhat
## bK_SL_bar	1.20842702	0.064627795	1.103811078	1.30933256	1.0030299
## bK_S_bar	0.93777882	0.170713615	0.661447332	1.21124660	1.0004919
## aK_bar	-2.15452403	0.048664137	-2.230824904	-2.07540921	1.0006849
## sigma_bK_SL	0.12589925	0.113162796	0.007867592	0.34112455	1.0004317
## sigma_bK_S	0.15312746	0.148318187	0.008917186	0.43679075	1.0001233
## sigma_aK	0.11090959	0.096789470	0.007394145	0.29478525	1.0012947
## bK_dK	-0.20949502	0.011327227	-0.227626227	-0.19163364	1.0001809
## U	0.01529139	0.002785008	0.010865210	0.01984109	1.0047042
## aS_bar	-0.78543927	0.038527561	-0.844674766	-0.72296451	1.0000068
## sigma_aS	1.37987453	0.232789230	1.037964790	1.78292757	1.0011785
## bS_dS	-0.20639919	0.015941472	-0.231898878	-0.18115260	0.9998069
## sigma_S	0.04714914	0.002555594	0.043342483	0.05148746	1.0012626
## ess_bulk					
## bK_SL_bar	798.8358				
## bK_S_bar	3137.3252				
## aK_bar	1901.3698				
## sigma_bK_SL	4011.5825				
## sigma_bK_S	6623.7017				
## sigma_aK	2995.5071				
## bK_dK	3306.1387				
## U	468.5726				
## aS_bar	4769.8179				
## sigma_aS	4265.9277				
## bS_dS	16888.4446				
## sigma_S	12205.4498				

- This is very high eff and no divergent. This is great. This model is reliably sampled to be used for

analysis!

Posterior Check

```
post <- extract.samples(m1.3)
```

Hidden parameters

- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y, main = "Difference between original (black) and simulated Y (red)")
points(1:200, post$Y[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

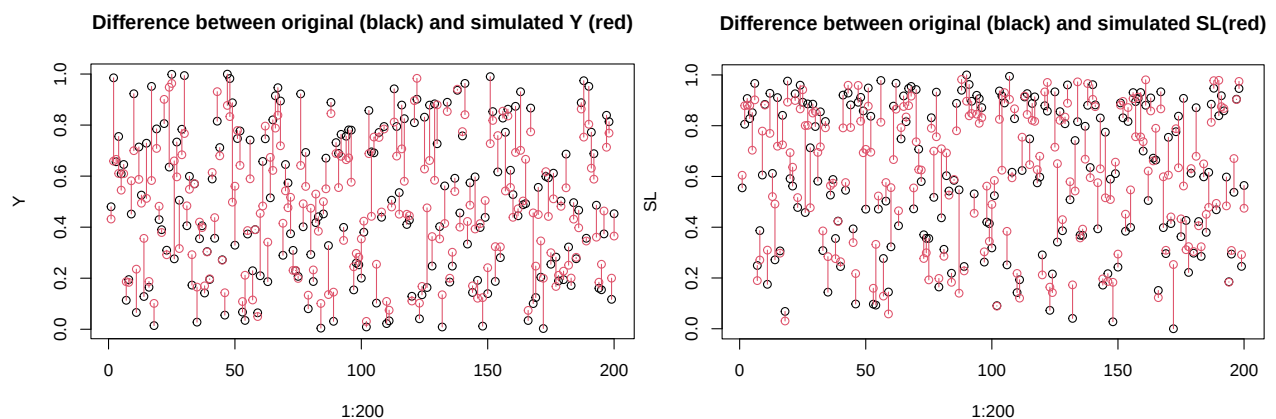
plot(1:200, SL, main = "Difference between original (black) and simulated SL(red)")
points(1:200, post$SL[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$SL[1,i], SL[i]), col = 2)

# check Y and SL with K
plot(Y, d$K, main = "Difference between original (black) and simulated Y (red) on d$K")
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

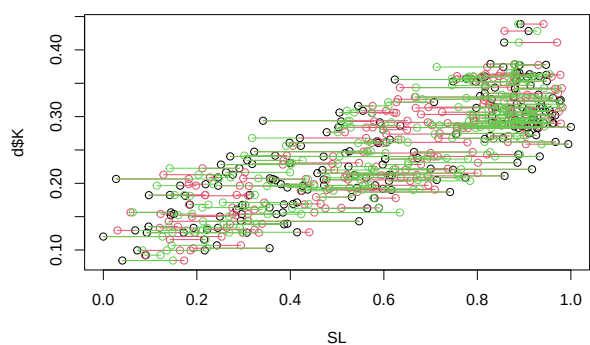
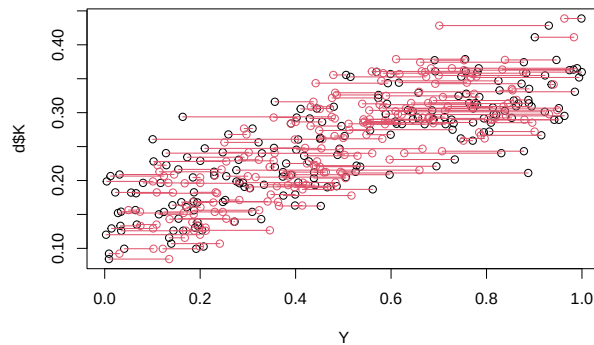
plot(SL, d$K, main = "Difference between original (black) and simulated SL (red and green) on d$K")
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

# Y and T
plot(Y, d$T, main = "Difference between original (black) and simulated Y (red) on d$T")
points(post$Y[1,], d$T, col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i], d$T[i]), col = 2)

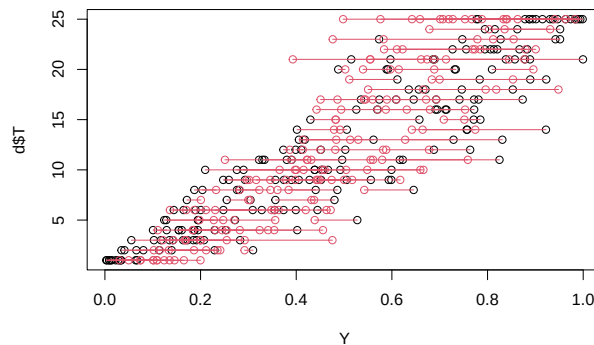
# Y and SL both hidden parameters
plot(Y, SL, main = "Difference between original (black) and simulation (red) of Y and SL")
points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)
```



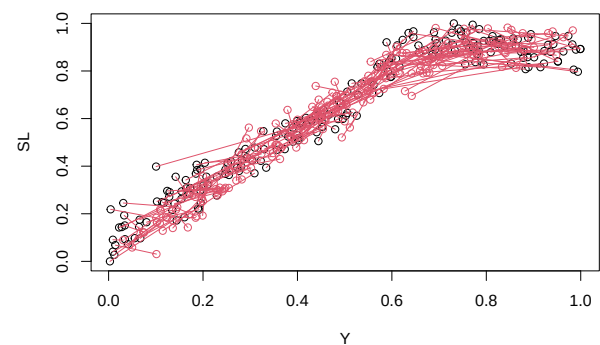
Difference between original (black) and simulated Y (red) on d\$K difference between original (black) and simulated SL (red and green) or



Difference between original (black) and simulated Y (red) on d\$T



Difference between original (black) and simulation (red) of Y and SI



=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthesized) data.

Posterior Simulation

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- apply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function(i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- apply(SLseq,
  function(i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$S)
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

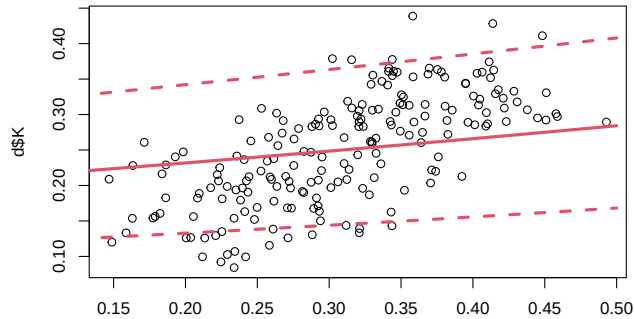
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
```

```

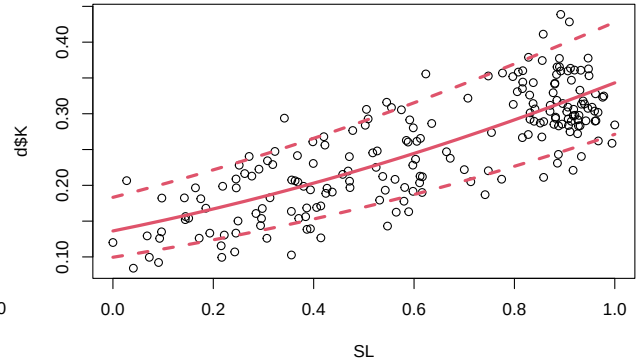
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```

Synthetic Data Plot(d\$S and d\$K)

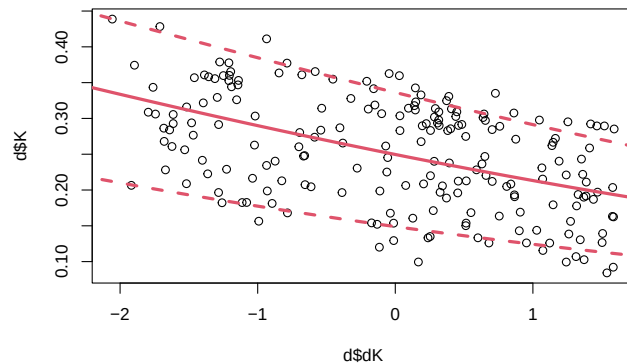


Synthetic Data Plot(SL and d\$K)



Model K

Synthetic Data Plot(d\$dK and d\$K)



- Looks nice!

Model S

- Do the same thing for S
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided in the data.

```
post$SL[1,] == post$S[1,]
```

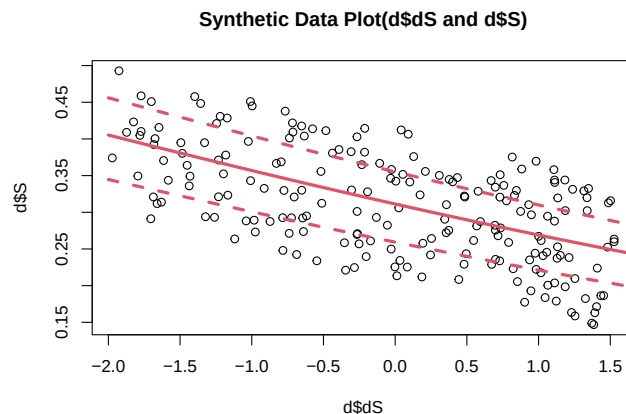
```

## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE

```

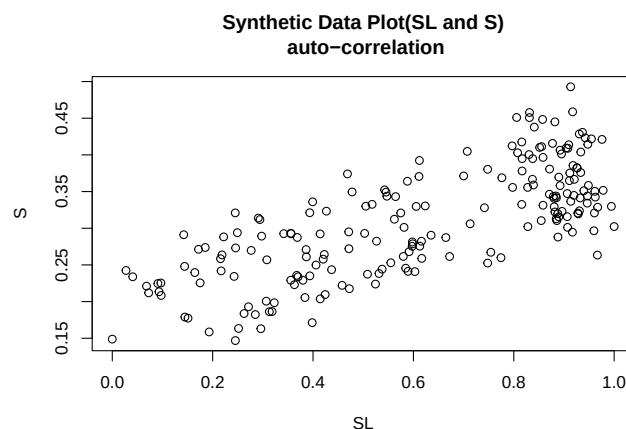
```
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function(i) inv_logit(post$aS + rep(post$bS_dS, 25) * i)) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



- Functions catch the trend very well!
- The model 1.3 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.
- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

```
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")
```



This is because there is a correlation!!

Model 2 (m2)

- see priors in overview.pdf, but this include correlation between SL and S
- This is based on the Ch 14.3 in Statistical Rethinking as SL and S are not centred

- warnings are annoying but relatively few so I just omit it for m2.2 and m.3.

```
# Model 2
m2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # We do not know anything about each parameter but think similar for all group.
    # So use partial pooling to regularize estimates.
    # mean should start at 0 and has small variation.

    # In this case (m2), we do include correlation between S and SL due to Y.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_Y_bar)~normal(0,0.3),
    c(sigma_aK, sigma_bK_Y) ~ exponential(6),
    z_aK[T]~normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],

    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1), # I do not expect very very strong corre
    matrix[2,25]: zSSL ~ normal(0 , 1),

    # don't need regularization
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

  # Model S
  S ~ normal(mu_S, sigma_S),
  logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,

  # I am hoping/believing b_dS is negative and b_Y_S is positive
  # partial pooling for the same reason for a2 and b_Y_S
  vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
  vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
  c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
  c(sigma_aS, sigmabS_Y) ~ exponential(6),
  z_aS[T] ~ normal(0,0.1),
  zbS_Y[T] ~ normal(0,0.1),

  # don't need regularization
  bS_dS ~ normal(0,0.3),
  sigma_S ~ exponential(1),

  # Model SL by Y
  vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1789:3: note: by 'void ulam_cmd
## 1789 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1789:3: note: by 'void ulam_cmd
## 1789 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1828:3: n
## ote: by 'T_ ulam_cmdstanr_faadbf2fa06d1ae2933d92a379944cca_model_namespace::ulam_cmdstanr_faadbf2f
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1828:3: note: by 'T_ ulam_cmdsta
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1828:3: note: by 'T_ ulam_cmdsta
## :ulam_cmdstanr_faadbf2fa06d1ae2933d92a379944cca_model::log_prob(Eigen::Matrix<T_a, -1, 1>&, std::ost
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-5040399cd6b.hpp:1828:3: note: by 'T_ ulam_cmdsta
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~

```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)

```

```

## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)

```

```

## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)

```



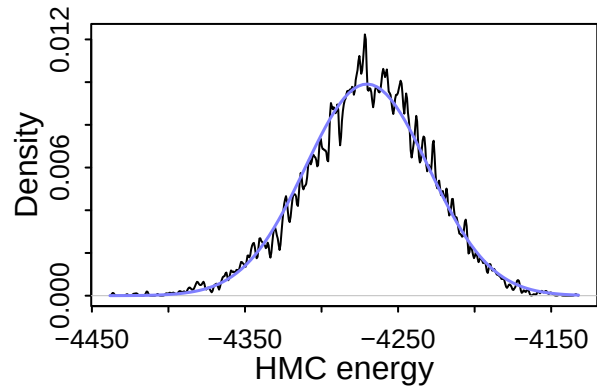
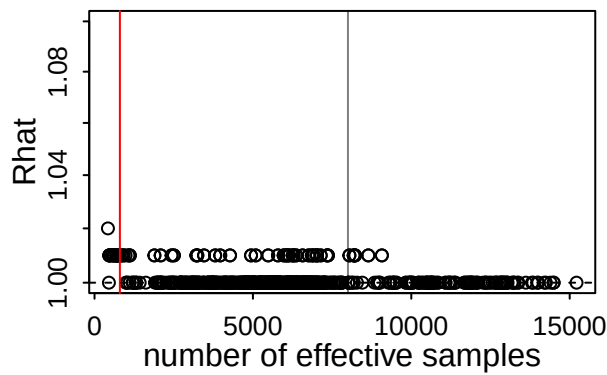
```

## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 140.8 seconds.
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 171.8 seconds.
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 181.9 seconds.
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 1782.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 569.3 seconds.
## Total execution time: 1782.9 seconds.

## Warning: 46 of 8000 (1.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

```

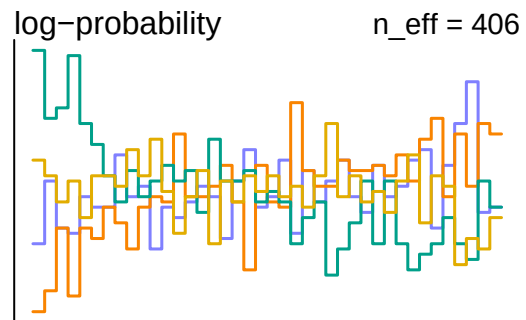
[dashboard\(m2\)](#)



46

Divergent transitions

Check yourself before
you wreck yourself



-> Looks bad sampling efficiency.

Therefore, I will tackle this problem with the same approach though I do not want to remove the correlation parameters.

```
# Model 2,2
m2.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    aK_bar~normal(0,0.3),
    sigma_aK ~ exponential(6),
    z_aK[T]~normal(0,0.1),

    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],

    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
    matrix[2,25]: zSSL ~ normal(0 , 1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
```

```

c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
c(sigma_aS, sigmabS_Y) ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
zbS_Y[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:  700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:  500 / 4000 [12%] (Warmup)

```

```

## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)

```

```

## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 250.0 seconds.
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)

```

```

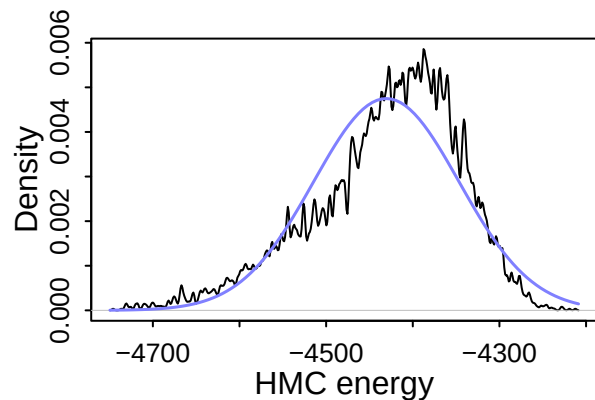
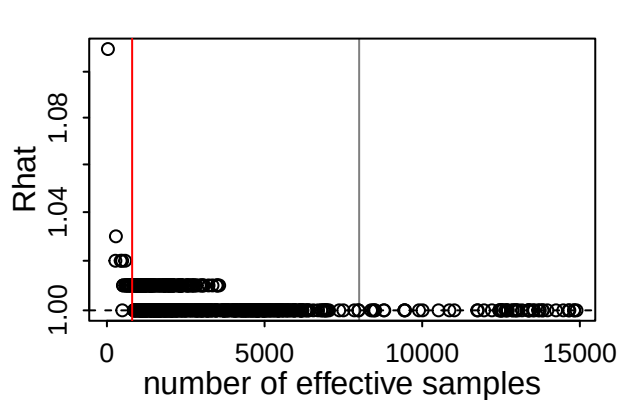
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 290.9 seconds.
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 335.1 seconds.
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 415.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 322.7 seconds.
## Total execution time: 415.1 seconds.

```

```

dashboard(m2.2)

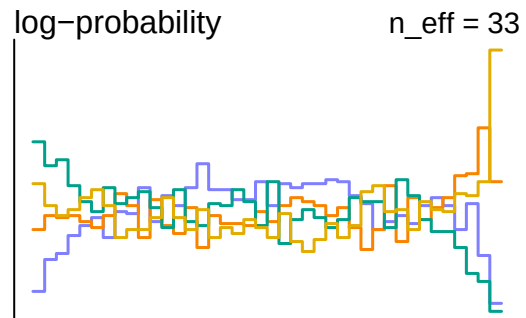
```



0

Divergent transitions

Outlook good



- This sampling efficiency is still not great. Let me remove one more thing so that it corresponds m1.3.

```
# Model 2.3
m2.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    aK_bar ~ normal(0,0.3),
    sigma_aK ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],
    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
    matrix[2,25]: zSSL ~ normal(0 , 1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_dS * dS ,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    aS_bar ~ normal(0,0.3),
    sigma_aS ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    bS_dS ~ normal(0,0.1),
    sigma_S ~ exponential(1),
```

```

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
```

```
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   900 / 4000 [22%] (Warmup)

```



```

## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)

```

```

## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 483.7 seconds.
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)

```

```

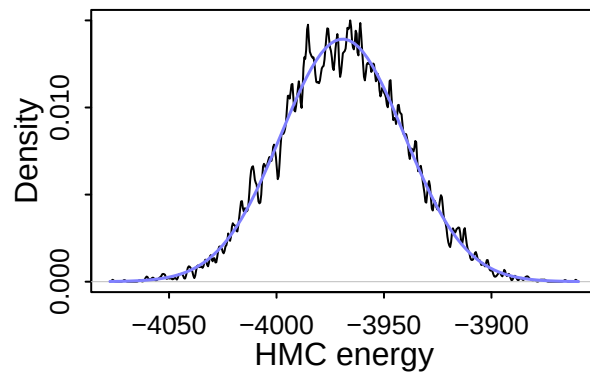
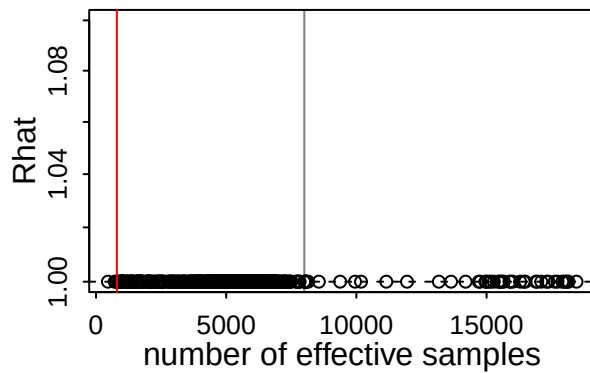
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 497.2 seconds.
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 517.5 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 521.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 504.9 seconds.
## Total execution time: 521.5 seconds.

```

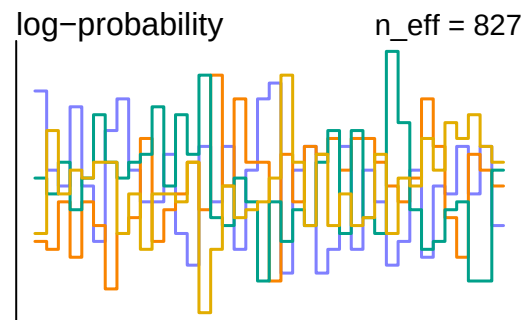
```

dashboard(m2.3)

```



0
Divergent transitions
Outlook good



```
precis(m2.3, depth=1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
aK_bar	-1.39342498	0.076023640	-1.516231624	-1.2735773	1.001434	3392.5777
sigma_aK	0.18550931	0.194430729	0.009801344	0.5479671	1.000086	7772.4919
bK_dK	-0.20606640	0.012574920	-0.225552160	-0.1861764	1.000053	11149.2278
U	0.02776627	0.002502351	0.023781422	0.0317243	1.001953	715.2716
aS_bar	-0.78501742	0.038300019	-0.844601082	-0.7220154	1.001650	2930.6319
sigma_aS	1.38286095	0.233883064	1.036751168	1.7770679	0.999921	3971.2467
bS_dS	-0.20204174	0.015381664	-0.226717019	-0.1770867	1.000736	16893.5471
sigma_S	0.04716123	0.002466717	0.043435937	0.0512440	1.000917	11945.2473

- This model is good enough.

Posterior Check

```
post <- extract.samples(m2.3)
```

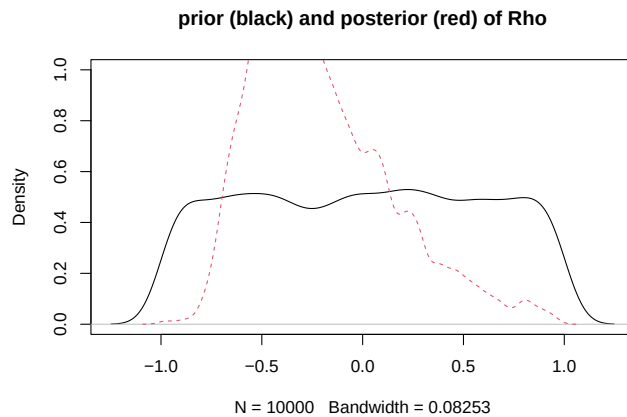
- I do not check the Hidden parameters since I have never touched it since m1

Correlation?

- Rho value shows a bit of correlation between S and SL.

```
R <- rlkjcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1), main = "prior (black) and posterior (red) of Rho")
# This is complicated as cholesky correlation
true_rho_samples=NULL
for (i in 1:2000) {
  L <- post$RhoSSL[i, , ]
  R <- L %*% t(L)
  true_rho_samples[i] <- R[1, 2]
```

```
}
dens(true_rho_samples, add = TRUE, lty = 2, col = 2)
```



Posterior Predictions

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$d
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

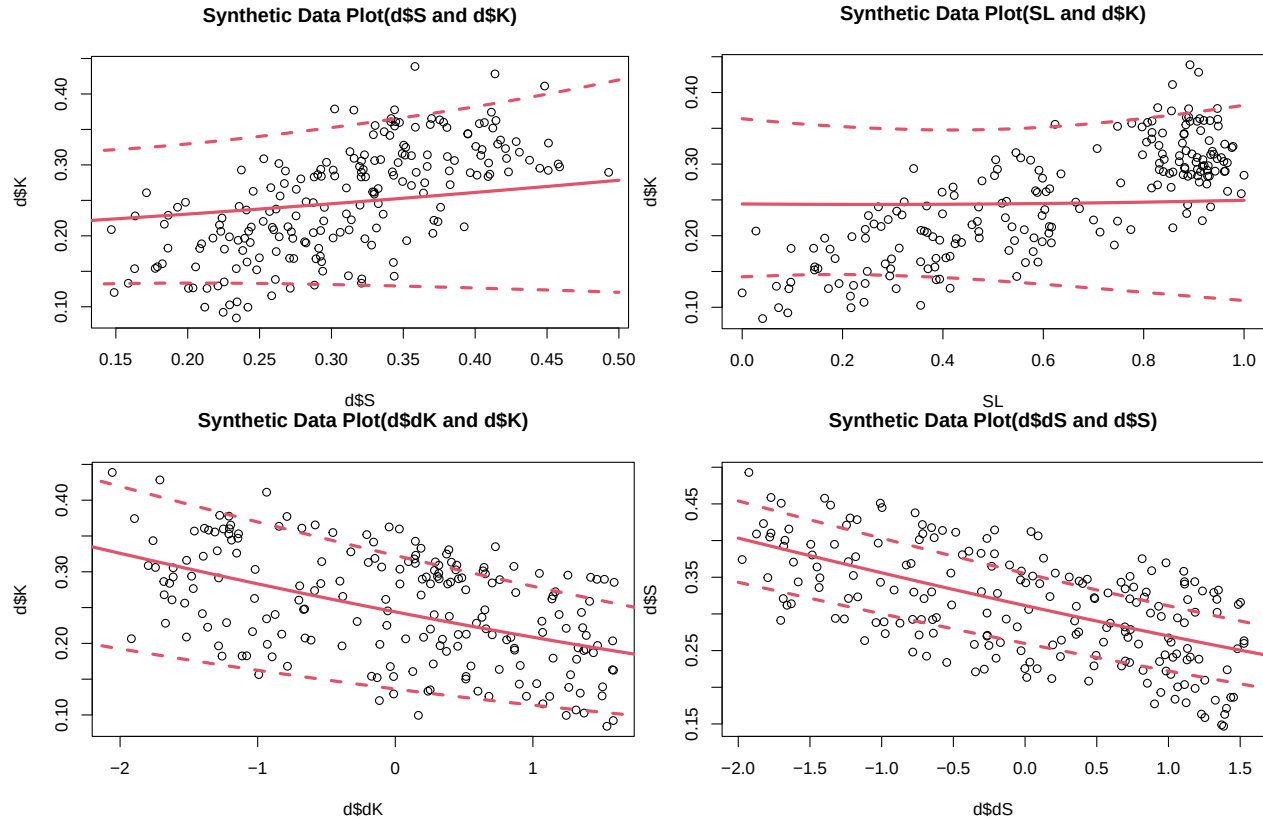
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
```

```

PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```



Overall K estimation is fine, but fit less than m1.3.

Let me compare those two models

Compare Models

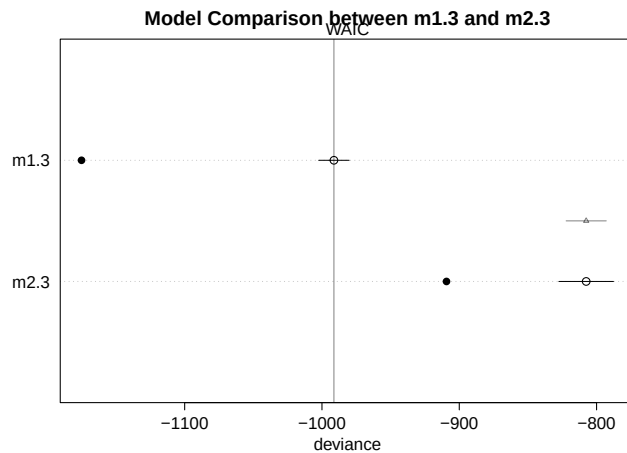
```

# Compare models
model_comparison <- compare(m1.3, m2.3)
print(model_comparison)

##           WAIC      SE    dWAIC    dSE   pWAIC      weight
## m1.3 -991.2934 11.10711  0.0000    NA  91.91607 1.000000e+00
## m2.3 -807.5617 19.90347 183.7316 14.6204 50.87370 1.268185e-40

# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m2.3")

```



Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

=> Therefore, the m1 is better than the other one.

Model 3

- Same thing! See Overview pdf for details.

```
m3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],

    # Basically same as m1 but not regularization (no pooling).
    # bK_T is regularized to maintain the efficiency.

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.1), # This is different from the other one, so th
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1), # regularization but not high sigma
    sigma_bK_T ~ exponential(1), # Needs to be 1 to maintain bK_T.
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

    # I am hoping/believing b_dS is negative, b_Y_S is positive, and bS_T is different for each one
    c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
    # bS_T is regularized.
```

```

    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
    T ~ poisson(Y * 25),

    # Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1308:3: note: by 'void ulam_cm
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1308:3: note: by 'void ulam_cm
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note: by 'T_ ulam_cmds
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note: by 'T_ ulam_cmds
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note: by 'T_ ulam_cmds

```



```

## const'
## 1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_crtp<T_>::log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const' defined but not used [-Wunused-function]
## 159 |   inline math::var log_prob(std::vector<math::var>& theta,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note:   by 'T_ ulam_cmds'
## 1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note:   by 'T_ ulam_cmds')
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be either severely ill-conditioned or misspecified.
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note:   by 'T_ ulam_cmds')
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be either severely ill-conditioned or misspecified.
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 2
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note:   by 'T_ ulam_cmds')
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be either severely ill-conditioned or misspecified.
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 3
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the following reasons:
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/Rtmp2dglvq/model-504022c93996.hpp:1345:3: note:   by 'T_ ulam_cmds')
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, then the sampler may run fine, but the model may be either severely ill-conditioned or misspecified.
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 4
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)

```

```

## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)

```

```

## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)

```

```

## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 167.6 seconds.
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 finished in 173.6 seconds.
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 177.6 seconds.
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 182.9 seconds.

```

```
##
## All 4 chains finished successfully.
## Mean chain execution time: 175.4 seconds.
## Total execution time: 183.0 seconds.
```

```
precis(m3, depth = 1)
```

```
## 450 vector or matrix parameters hidden. Use depth=2 to show them.
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
bK_dK	-0.21059878	0.009511628	-0.225458272	-0.19511320	0.9999813	9647.646
bK_Y	0.55272960	0.072294607	0.435250900	0.66726275	1.0001738	6990.647
bK_SL	0.52594543	0.072928727	0.408581937	0.64286910	1.0003291	5855.309
bK_S	0.20038723	0.092202521	0.054469078	0.34873690	1.0001477	7610.834
aK	-0.20029161	0.106808086	-0.370280048	-0.03107452	0.9998425	7208.708
bK_T_bar	-0.19877544	0.105782879	-0.368025955	-0.03122630	1.0004872	7488.179
sigma_bK_T	10.96851454	1.782497923	8.285063962	13.96272726	1.0013552	1282.372
U	0.02041013	0.001597735	0.017884095	0.02298565	1.0004373	2629.516
bS_dS	-0.21973289	0.007200949	-0.231202317	-0.20805002	1.0007128	2256.162
bS_Y	0.94932607	0.035192968	0.892037489	1.00471177	1.0028021	1224.265
aS	-0.63334197	0.070996333	-0.747435827	-0.52136775	1.0002181	7010.610
bS_T_bar	-0.63336506	0.071610524	-0.747797037	-0.51850441	1.0007631	6852.737
sigma_bS_T	0.53197839	0.173693046	0.299727333	0.83009712	1.0009731	1709.613
sigma_S	0.01131049	0.001756825	0.008460197	0.01407877	1.0020314	851.170

- without divergent but it has a few problem with efficiency. I will try to eliminate by removing parameters.

```
m3.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

    c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T
  T ~ poisson(Y * 25),

# Model T by Y
  vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL   = "lower=0,upper=1",
  Y    = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:  1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)

```

```

## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)

```

```

## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)

```



```

## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 175.0 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 178.7 seconds.
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 180.4 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 183.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 179.5 seconds.
## Total execution time: 184.0 seconds.

```

```
precis(m3.2, depth = 1)
```

	mean	sd	5.5%	94.5%	rhat
## bK_dK	-0.2054233815	0.010094859	-0.221543138	-0.18938009	1.0005707
## bK_Y	-0.0004158155	0.099234856	-0.157870633	0.15945061	1.0000662
## bK_SL	0.9308080367	0.052287203	0.845587266	1.01215919	1.0002817
## bK_S	0.2698265391	0.092529312	0.118594790	0.41673836	0.9998186
## aK	-0.2001936704	0.105789342	-0.371793095	-0.03285090	1.0000903
## bK_T_bar	-0.2006305049	0.106943438	-0.369229993	-0.02943072	1.0004117
## sigma_bK_T	11.0222976108	1.823808881	8.320404843	14.06954913	1.0022271
## U	0.0220538058	0.001862367	0.019205088	0.02514622	1.0003783
## bS_dS	-0.2266198632	0.007414163	-0.238269071	-0.21458477	1.0009648
## bS_Y	0.9604646396	0.035941033	0.902832554	1.01708941	1.0016360
## aS	-0.6347843806	0.072157912	-0.750790724	-0.51945046	1.0010274
## bS_T_bar	-0.6324945202	0.071270253	-0.745690814	-0.51798972	1.0004800
## sigma_bS_T	0.5394693083	0.173420306	0.302973500	0.84608175	1.0011213
## sigma_S	0.0105295496	0.001864119	0.007527274	0.01343426	1.0029627
##	ess_bulk				
## bK_dK	11681.818				
## bK_Y	16419.843				
## bK_SL	4666.966				
## bK_S	11502.937				
## aK	7543.805				

```
## bK_T_bar      8336.282
## sigma_bK_T    1529.569
## U             3461.654
## bS_dS         1897.722
## bS_Y          887.124
## aS            8729.802
## bS_T_bar      9865.451
## sigma_bS_T    1816.940
## sigma_S       566.222
```

- This is still not the best model with low efficiency. I will remove one more parameter.
- I can remove bK_T or bS_T but this time, I want to make this similar to m1.3 and m2.3.

```
m3.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_dK) ~ normal(0,0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_T[T],

    c(aS, bS_dS) ~ normal(0,0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
    T ~ poisson(Y * 25),

    # Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
  ), dat = d, constraints = list(
    SL = "lower=0,upper=1",
    Y = "lower=0,upper=1"
  ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 1 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:    900 / 4000 [22%] (Warmup)
## Chain 4 Iteration:    900 / 4000 [22%] (Warmup)
## Chain 2 Iteration:    900 / 4000 [22%] (Warmup)
## Chain 3 Iteration:   1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration:    900 / 4000 [22%] (Warmup)
## Chain 4 Iteration:   1000 / 4000 [25%] (Warmup)
## Chain 2 Iteration:   1000 / 4000 [25%] (Warmup)
## Chain 3 Iteration:   1100 / 4000 [27%] (Warmup)
## Chain 1 Iteration:   1000 / 4000 [25%] (Warmup)
## Chain 4 Iteration:   1100 / 4000 [27%] (Warmup)
## Chain 2 Iteration:   1100 / 4000 [27%] (Warmup)
## Chain 3 Iteration:   1200 / 4000 [30%] (Warmup)
## Chain 1 Iteration:   1100 / 4000 [27%] (Warmup)
## Chain 4 Iteration:   1200 / 4000 [30%] (Warmup)

```

```

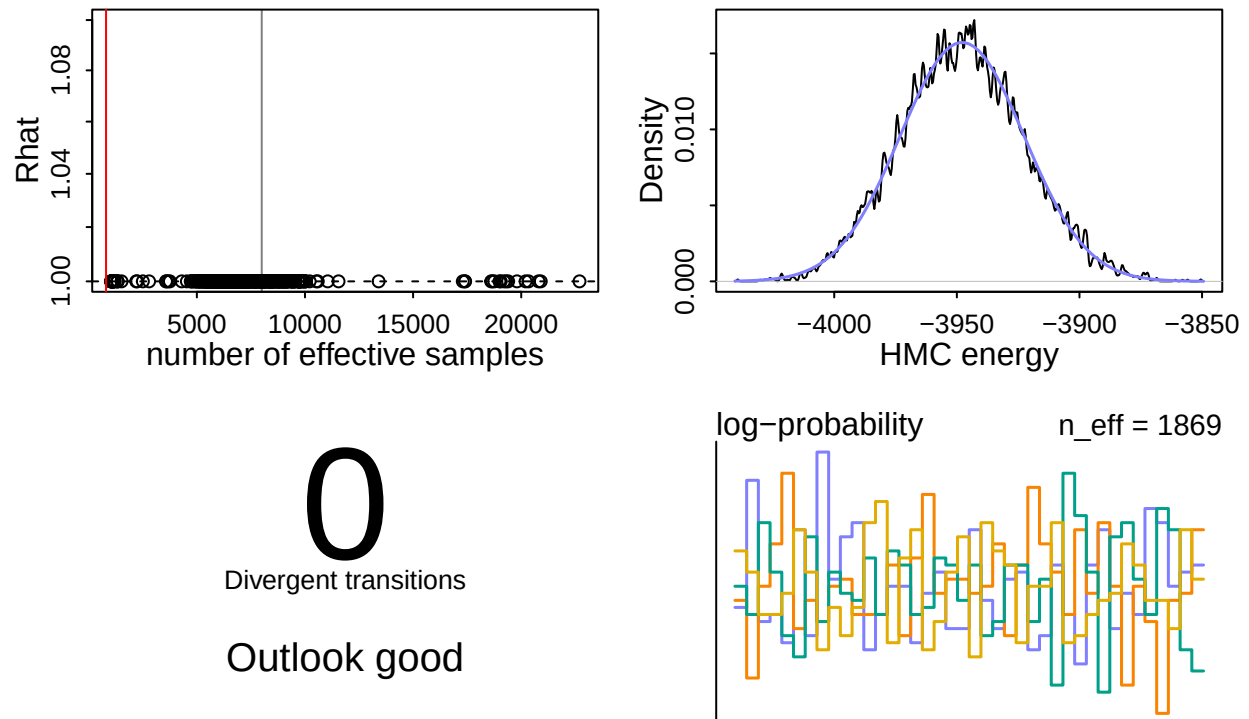
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)

```

[illegible]

```
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 82.8 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 84.1 seconds.
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 84.7 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 85.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 84.3 seconds.
## Total execution time: 86.0 seconds.
```

```
dashboard(m3.3)
```



```
precis(m3.3, depth = 1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
## bK_dK	-0.19096112	0.014915920	-0.21492099	-0.16687789	1.000390	11551.951
## bK_SL	0.66429684	0.087345959	0.52140651	0.80334631	1.000834	2501.704
## bK_S	0.27377147	0.098613464	0.11671509	0.43091436	1.000624	11052.353
## aK	-0.22527291	0.111171012	-0.40515750	-0.05108245	1.000667	4955.455
## bK_T_bar	-0.22518624	0.108828343	-0.40222881	-0.05450119	1.000457	4884.464
## sigma_bK_T	9.50586383	1.854975614	6.72042152	12.58480510	1.001102	1516.920
## U	0.03426782	0.003258108	0.02914756	0.03963545	1.000465	2241.956
## bS_dS	-0.20119370	0.015343550	-0.22558199	-0.17644210	1.000638	20201.039

```
## aS          -0.34727498 0.079239923 -0.47333928 -0.22061450 1.000562 7989.696
## bS_T_bar    -0.34970605 0.079811577 -0.47549591 -0.21917713 1.000342 8143.489
## sigma_bS_T  2.06431848 0.504423967 1.40338302 2.96659654 1.001480 2195.825
## sigma_S      0.04685314 0.002494249 0.04306726 0.05105682 1.000671 13404.588
```

Posterior Check

```
post <- extract.samples(m3.3)
```

- Again, I do not check the Hidden parameters since I have never touched it since m1

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
  function(i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK)
), mean)
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
points(post$SL[1,], d$K, col = 2) # include the estimated points, though we confirmed above that it is
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_dK * d$dK + colMeans(post$SL))
), mean)
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * i + colMeans(post$dK))
), mean)
PIs<- apply( KbydK , 2 , PI )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
KbyT<- sapply(1:25,
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK)
), mean)
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)
```

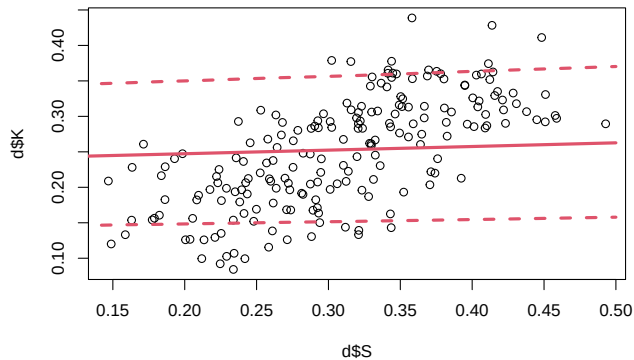
```

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + post$bS_dS * i + colMeans(post$bS_T)[d$T] )) # Used mean of each s
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

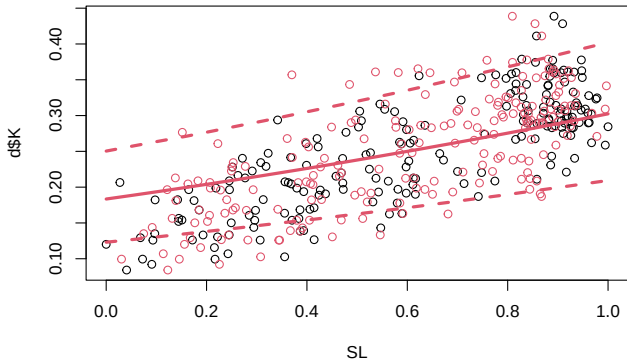
plot(T, d$S, main = "Synthetic Data Plot(d$T and d$S)")
SbyT<- sapply(1:25,
  function (i) inv_logit(post$aS + post$bS_dS * d$dS + colMeans(post$bS_T)[i]))
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)

```

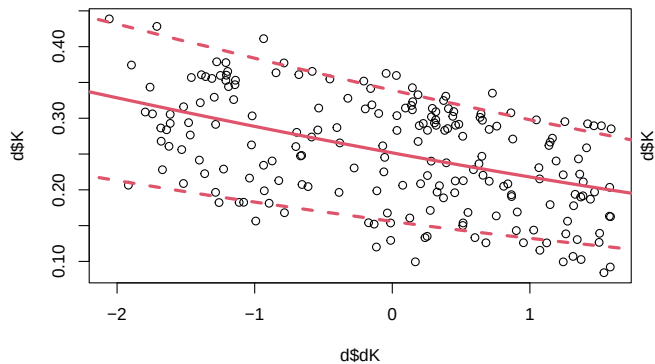
Synthetic Data Plot(d\$S and d\$K)



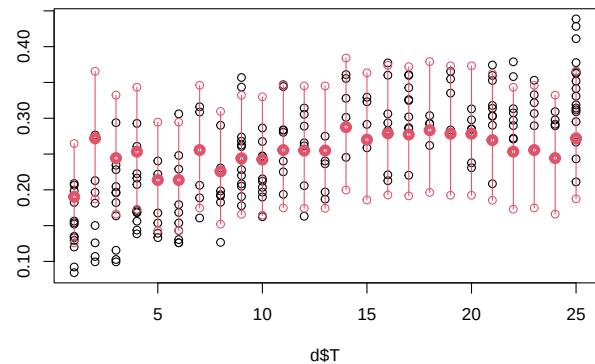
Synthetic Data Plot(SL and d\$K)

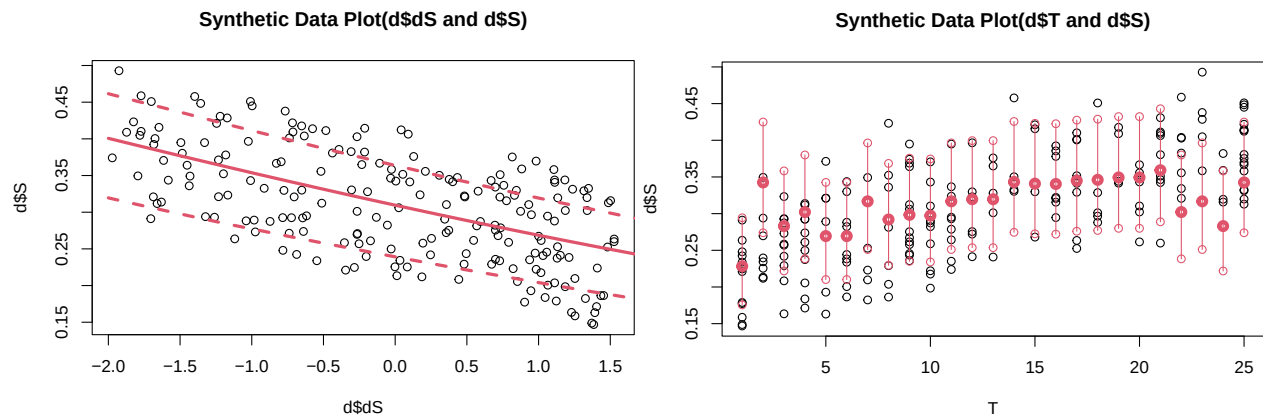


Synthetic Data Plot(d\$dK and d\$K)



Synthetic Data Plot(d\$T and d\$K)





I have no idea why it does not model bK_T and bS_T very well... I will figure it out. Other than that, looks great, let me compare all those models.

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m3.3)
print(model_comparison)
```

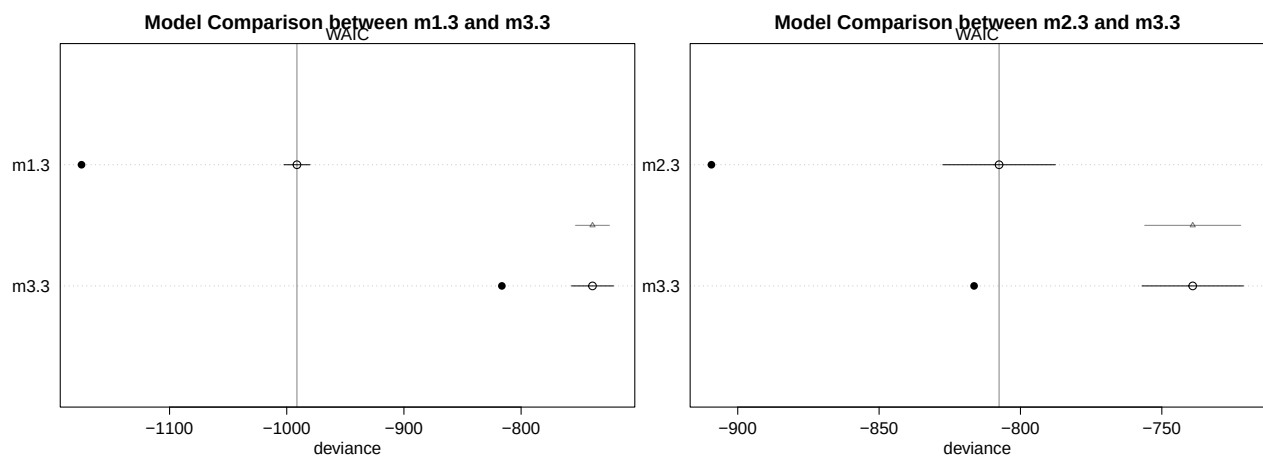
	WAIC	SE	dWAIC	dSE	pWAIC	weight
m1.3	-991.2934	11.10711	0.0000	NA	91.91607	1.000000e+00
m3.3	-739.0765	17.93850	252.2168	14.48927	38.66522	1.705344e-55

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m3.3")
```

```
model_comparison <- compare(m2.3, m3.3)
print(model_comparison)
```

	WAIC	SE	dWAIC	dSE	pWAIC	weight
m2.3	-807.5617	19.90347	0.00000	NA	50.87370	1.000000e+00
m3.3	-739.0765	17.93850	68.48519	16.98636	38.66522	1.344712e-15

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m2.3 and m3.3")
```



This looks Model 1 is a bit better in this case.

Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K) may be examined with actual data.
- The Model 3 still has a problem of bK_T and bS_T as they do not fit the data. I am trying to understand the reason and fix it.

```
rm(list=ls())
```